Vulnerability Notes

Local file inclusion (LFI)

Concepts, Attack Techniques, Detection, and Defenses

Prepared by
Youssef Reda

Contents

Local File Inclusion (LFI) **1**
Overview . . . 1
How PHP `include()` Handles Files . . . 1
How LFI Can Lead to a Breach . . . 1
How LFI Attacks Work in Real Applications . . . 2
User-Controlled File Paths . . . 2
Preventing LFI with an Allowlist . . . 3
LFI Beyond PHP . . . 3

LFI Exploitation **4**
Exploitation: Path Manipulation . . . 4
Path Traversal . . . 4
Null Byte Injection . . . 4
PHP Wrapper Abuse . . . 4
Reconnaissance: Sensitive File Disclosure . . . 4
Escalation: Chaining LFI with Other Vulnerabilities . . . 5
Sensitive Data Exposure . . . 5
LFI to Cross-Site Scripting (XSS) . . . 5
LFI to Remote Code Execution (RCE) . . . 5
High-Value Targets in LFI . . . 5

LFI vs. Related Vulnerabilities **6**
LFI vs. Remote File Inclusion (RFI) . . . 6
LFI vs. Path Traversal . . . 6

Real-World Impact of LFI Vulnerabilities **7**
Historical Example: TimThumb . . . 7

LFI Prevention and Detection **8**
How to Prevent LFI Vulnerabilities . . . 8
How to Detect LFI Vulnerabilities . . . 8
How Automated Scanners Detect LFI . . . 8

Local File Inclusion (LFI)

Overview

LFI is a web vulnerability that occurs when an application uses insufficiently validated user-controlled input to select and include a local file.

For example, in PHP:

```php
<?php
$page = $_GET['page'];
include($page);
?>
```

The application passes the user-controlled `page` parameter directly to `include()`, potentially allowing unintended files to be included.

How PHP `include()` Handles Files

The `include()` function is not restricted to PHP files. It can include any readable file, but the resulting behavior depends on the file's contents and how PHP processes it.

- **PHP files:** PHP code is executed when included.
- **Plain-text files:** Their contents are generally treated as output rather than executed as PHP code.
- **Binary files:** Their contents may produce unexpected or unusable output; they are not automatically executed as PHP code.

The file extension alone does not determine whether a file can be included. File accessibility, PHP's processing behavior, and application-specific restrictions also matter.

How LFI Can Lead to a Breach

Local File Inclusion (LFI) can expose sensitive files on a web server when an application uses insufficiently validated user input to determine which files to load.

The impact can extend beyond file disclosure:

1. **File Disclosure:** Access sensitive files such as `/etc/passwd`.
2. **Credential Exposure:** Discover database credentials, API keys, or tokens in configuration files such as `.env`.
3. **Unauthorized Access:** Abuse exposed credentials to access other services, provided the credentials are valid and the services are reachable.
4. **Further Compromise:** Chain LFI with other weaknesses to potentially achieve Remote Code Execution (RCE) or compromise additional systems.

These outcomes depend on file permissions, application behavior, and the availability of additional attack paths. LFI does not automatically lead to RCE.

How LFI Attacks Work in Real Applications

User-Controlled File Paths

LFI can occur when an application uses user-controlled input to determine which file to include or access without proper restrictions.

Common input sources include:

- Query parameters and form inputs.
- Cookies.
- HTTP headers, such as `User-Agent`.
- Other user-controlled values used in file selection.

Example of vulnerable PHP code:

```
<?php
$file = $_GET['file'];
include($file);
?>
```

The application passes the user-controlled `file` parameter directly to `include()`, potentially allowing unintended files to be included. User-controlled input alone does not constitute a vulnerability; the risk arises when it influences file selection without adequate restrictions.

Preventing LFI with an Allowlist

A safer approach is to map predefined page identifiers to specific file paths rather than allowing users to supply arbitrary paths.

```php
<?php
$allowed = [
    'home'  => 'home.php',
    'about' => 'about.php'
];

$page = $_GET['page'] ?? 'home';

if (array_key_exists($page, $allowed)) {
    include(__DIR__ . '/pages/' . $allowed[$page]);
} else {
    http_response_code(404);
}
?>
```

- `$allowed`: Maps permitted page identifiers to predefined filenames.
- `array_key_exists()`: Checks whether the requested identifier is allowlisted.
- `__DIR__`: Provides the directory of the current PHP file, helping construct a predictable path.
- Invalid identifiers are rejected with an HTTP 404 response.

This approach prevents users from directly specifying arbitrary file paths. The mapped files and their directories must still be protected against unauthorized modification.

LFI Beyond PHP

Similar vulnerabilities can occur in other languages when user-controlled input determines which file is accessed.

Examples include:

- **PHP:** `include()`.
- **Python:** File-serving functions such as `send_file()`.
- **Node.js:** File-system functions such as `fs.readFile()`.
- **Java:** Resource-loading mechanisms.

Using these functions does not automatically introduce LFI. The vulnerability depends on how file paths are constructed, validated, and restricted.

LFI Exploitation

Exploitation: Path Manipulation

Attackers exploit LFI by manipulating user-controlled file paths to access unintended files or alternative data sources.

Path Traversal

The `../` sequence moves up one directory level and may allow access outside the intended directory.

```
?page=../../../../etc/passwd
```

Encoded traversal sequences may bypass inadequate filters. For example, `%2e%2e%2f` represents `../` after URL decoding.

Null Byte Injection

The `%00` sequence represents a null byte. Older PHP versions (before PHP 5.3.4) could interpret null bytes as string terminators in file paths, enabling certain extension-validation bypasses. This is a legacy technique and does not generally apply to modern PHP versions.

PHP Wrapper Abuse

PHP wrappers can change how an inclusion source is accessed or processed.

```
?file=php://filter/convert.base64-encode/resource=index.php
```

The `php://filter` wrapper can Base64-encode a file's contents, potentially exposing PHP source code without executing the original code during inclusion. The `php://input` and `data://` wrappers can provide alternative data sources. Under specific conditions, including suitable PHP configuration and unsafe inclusion behavior, they may contribute to Remote Code Execution (RCE). Basic sanitization and blocklists are insufficient as standalone defenses because alternative encodings and path representations may bypass them. Prefer strict allowlists and controlled file mappings.

Reconnaissance: Sensitive File Disclosure

Once unintended file access is established, attackers may investigate files that reveal application details, credentials, or runtime information.

- `/etc/passwd`: Local account information on Linux.
- `/var/www`: Application source code and related files, depending on the deployment.
- `.env`: Potential database credentials, API keys, and other secrets.
- `/var/log/apache2/access.log`: HTTP access logs that may contain user-controlled input.
- `/var/log/apache2/error.log`: Server and application error information.
- `/proc/self/environ`: Environment variables available to the current process, potentially including secrets.

File accessibility depends on operating-system permissions, the web application's privileges, and the deployment environment.

Escalation: Chaining LFI with Other Vulnerabilities

LFI may be combined with other weaknesses to increase its impact.

Sensitive Data Exposure

Reading configuration files may reveal credentials or tokens that could enable unauthorized access to other services if they remain valid and are usable by the attacker.

LFI to Cross-Site Scripting (XSS)

Including attacker-controlled HTML or JavaScript in a response without appropriate output handling may lead to XSS. In this case, the code executes in the victim's browser rather than on the server.

LFI to Remote Code Execution (RCE)

RCE may be possible when LFI allows attacker-controlled content to be included and interpreted as PHP code.

Common chaining techniques include:

- **Log Poisoning:** Injecting attacker-controlled content into a writable log, then including that log through the vulnerable functionality. Execution requires the included content to be interpreted as PHP code.
- **File Upload Abuse:** Uploading a file containing attacker-controlled content and forcing the application to include it. This depends on upload permissions, file accessibility, and PHP's processing behavior.

LFI does not automatically result in RCE. Successful escalation requires additional conditions that allow attacker-controlled content to be executed.

High-Value Targets in LFI

Files that may provide useful information or enable further attacks include:

- **Environment and secrets files:** Database credentials, API keys, and cloud tokens.
- **Application configuration:** Connection strings, authentication secrets, and internal service endpoints.
- **Source code:** Hidden functionality, application logic, and additional vulnerabilities.
- **Logs and writable files:** Potential sources of attacker-controlled content for vulnerability chaining.
- **Cloud and container secrets:** Cloud credential files, mounted secrets, and Kubernetes service account tokens.

Exposure of cloud or container credentials may extend the impact beyond the web application, depending on the permissions associated with those credentials.

LFI vs. Related Vulnerabilities

LFI vs. Remote File Inclusion (RFI)

- **Local File Inclusion (LFI):** The application includes a file available on the local server.
- **Remote File Inclusion (RFI):** The application includes a file from an external source, provided the language, runtime configuration, and application behavior permit it.

RFI may allow an attacker to supply a malicious remote file, potentially resulting in Remote Code Execution (RCE). LFI can also lead to RCE when combined with additional weaknesses that enable attacker-controlled code execution.

LFI vs. Path Traversal

- **Path Traversal:** Manipulates file paths to access files outside the intended directory or access scope.
- **LFI:** Exploits application logic that includes or loads an unintended local file.

Example of path manipulation:

```
?file=../../etc/passwd
```

If the application reads and returns the file contents, the issue is generally classified as Path Traversal. If the input is passed to a file-inclusion mechanism such as PHP's `include()`, the issue may be classified as LFI. The vulnerabilities can overlap, but neither necessarily implies the other. Path Traversal does not require file inclusion or code execution, while LFI does not always require `../` sequences.

Real-World Impact of LFI Vulnerabilities

LFI vulnerabilities can have significant security and business consequences, depending on the files exposed and the possibility of chaining the vulnerability with other weaknesses.

- **Sensitive Information Disclosure:** Exposure of confidential data, application source code, and configuration files.
- **Credential Compromise:** Disclosure of database credentials, API keys, or tokens that may enable unauthorized access to other services.
- **Infrastructure Compromise:** Access to internal systems or cloud resources through exposed credentials and excessive permissions.
- **Remote Code Execution (RCE):** Potential server-side code execution when LFI is chained with additional weaknesses.
- **Business and Compliance Impact:** Data breaches may result in regulatory consequences, financial losses, incident response costs, and reputational damage.

Historical Example: TimThumb

TimThumb, a PHP image-processing library, suffered from historical vulnerabilities that enabled attackers to compromise websites through weaknesses in file handling and related exploitation techniques. This illustrates how a file-handling vulnerability can lead to broader compromise when combined with additional weaknesses. The actual severity of an LFI vulnerability depends on the demonstrated impact, accessible files, application permissions, and available exploitation paths. RCE and full server compromise must not be assumed without sufficient evidence.

LFI Prevention and Detection

How to Prevent LFI Vulnerabilities

- **Avoid Dynamic File Includes:** Do not use user input directly to determine which files to load. Map user selections to predefined resources.
- **Use Allowlists:** Restrict file selection to explicitly permitted values instead of relying on blocklists, which may be bypassed.
- **Use Indirect References:** Allow users to select internal identifiers rather than supplying filenames or paths directly.
- **Normalize and Validate Paths:** Canonicalize paths and verify that the resolved file remains within the permitted directory.
- **Restrict File Access:** Apply least-privilege permissions, separate writable directories from executable files, and consider PHP's `open_basedir` setting and operating-system isolation.
- **Patch Dependencies:** Update vulnerable libraries and frameworks. A Web Application Firewall (WAF) can provide additional protection but should not replace secure application logic.
- **Test Continuously:** Integrate automated scanning, code review, and authorized penetration testing into the Software Development Life Cycle (SDLC).

How to Detect LFI Vulnerabilities

- **Static Application Security Testing (SAST):** Analyzes source code to identify potentially unsafe data flows into file-handling functions, such as `include()`, `require()`, and `file_get_contents()` in PHP.
- **Software Composition Analysis (SCA):** Identifies vulnerable dependencies and versions associated with known CVEs.
- **Dynamic Application Security Testing (DAST):** Tests a running application using controlled inputs and analyzes responses for evidence of unintended file access.

Static analysis findings require contextual review. A potentially dangerous function does not automatically indicate an exploitable vulnerability.

How Automated Scanners Detect LFI

DAST scanners typically follow these steps:

1. **Crawling:** Discover application pages, parameters, and other accessible inputs.
2. **Payload Injection:** Submit test inputs designed to identify unintended local file access.
3. **Response Analysis:** Examine application responses for evidence that a local file was accessed.
4. **Verification:** Validate the evidence to reduce false positives and distinguish exploitable vulnerabilities from theoretical risks.

Automated detection coverage depends on application accessibility, authentication requirements, input validation, and the scanner's testing capabilities. Manual testing may still be necessary to identify missed cases.